

PROJECT REPORT

ON

MEDICINE REMINDER SYSTEM

**submitted in partial fulfillment of the requirements for the award of degree
of**

Bachelor of Computer Applications (BCA)



(Batch: 2025-2026)

UTTARANCHAL SCHOOL OF COMPUTING SCIENCES

UTTARANCHAL UNIVERSITY, DEHRADUN

PROJECT MENTOR:

Mr. Deepak Kumar Pathak
(Assistant Professor)

SUBMITTED BY:

Lata Negi
BCA(IV) E
Roll No. 22

ACKNOWLEDGEMENT

There are many people who helped directly or indirectly on the successful completion of my major project.

I am very thankful to our project guide “**Mr. Deepak Kumar Pathak**” who has been inspiring guide and committed caretaker for her unflinching devotion. The encouragement and support by her, especially in carrying out this project motivated me to complete this project.

I would like to thank Dean of USCS “**Prof Dr. Sonal Sharma**” for providing us with all the necessary resources required to complete the project. We profusely are thankful to the program head of the department “**Dr. Sameer Dev Sharma**” for their valuable guidance.

I would like to express deep sense of gratitude to our staff members of Dept. of Computer Applications for their co-operation, which has given in the congruency to build up this project. I would like to thank all our friends for their help and constructive criticism during my project period.

Finally, I am very much indebted to my parents for moral support and encouragement to achieve goals. I have no words to express my gratitude and still I am very thankful to my parents who have shown me this world and for every support they gave me.

Name : Lata Negi

Class : BCA (IV)E

Roll no. : 22

DECLARATION

I hereby declare that the project report entitled “**Medicine Reminder System**” submitted by **Lata Negi** to Uttaranchal School of Computing Science. The project was done under the Guidance of **Mr. Deepak Kumar Pathak** I further declare that the work reported in this project has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this university or any other university or institute.

Name: Lata Negi

Class: BCA(IV)E

Roll no.22

CERTIFICATE

This is to certify that the project entitled “**Medicine Reminder System**” by **Lata Negi** and has been submitted in the partial fulfillment of the requirements for the award of the degree of BCA from Uttarakhand University, Dehradun. The results embodied in this project have not been submitted to any other University or Institution for the record of any degree.

Under the guidance of:

Mr. Deepak Kumar Pathak

(Assistant Professor)

Uttarakhand School of Computing Sciences,

Uttarakhand University, Dehradun

TABLE OF CONTENT

Acknowledgement.....	ii
Declaration.....	iii
Certificate Of Originality.....	iv
1. Introduction	6
2. Objective.....	7
3. System Analysis.....	8—9
4. Gantt Chart.....	10
5. Pert chart.....	11
6. Software Requirement Specification (SRS).....	12
7. System Design	13-14
8. Testing.....	15-16
9. System Security.....	17-19
10. Problem Definition.....	20
11. Identification of Need	21
12. Feasibility Study	22
13. Project Planning	23
14. Flow Chart.....	24
15.Data Flow Diagram(DFD)	25
16. Scope	26
17. Code.....	27-33
18. Bibliography.....	34

1. INTRODUCTION

In the modern world, people are often busy with their daily routines, which makes it difficult to remember important tasks such as taking medicines on time. Missing doses or taking medicines at the wrong time can lead to serious health issues, especially for patients suffering from chronic illnesses or those who require regular medication. To overcome this problem, a Medicine Reminder System is developed using Java, which helps users manage their medication schedule efficiently and accurately.

The Medicine Reminder System is a simple Java-based application designed to remind users to take their medicines at the correct time. The system allows users to enter details such as the medicine name, dosage, number of times the medicine should be taken per day, and the starting time. Based on this input, the program calculates the appropriate intervals and generates a complete daily schedule. It can also display reminders or alerts at the specified times, ensuring that the user does not miss any dose.

The main objective of this project is to provide a reliable and user-friendly solution for medication management. By automating the reminder process, the system minimizes human error and improves consistency in taking medicines. This project is especially useful for elderly people, patients with busy schedules, and individuals who need to take multiple medicines throughout the day.

From a technical point of view, this project is developed using core Java concepts such as classes, objects, loops, conditional statements, and basic input/output operations. It may also use Java libraries like `java.util.Timer` or `java.time` for handling time and scheduling tasks. The system can be implemented as a console-based application or extended into a graphical user interface (GUI) using Java Swing or JavaFX for better user interaction.

Additionally, the project can be enhanced by integrating features like storing user data in files or databases, setting multiple reminders, tracking missed doses, and generating reports. These improvements can make the system more advanced and closer to real-world applications.

In conclusion, the Medicine Reminder System using Java is a practical mini project that addresses a common everyday problem. It not only helps users maintain a proper medication routine but also strengthens programming skills by applying Java concepts in a real-life scenario. This project is ideal for students and beginners who want to develop useful and meaningful software applications.

2. OBJECTIVE

1. To develop a program that allows users to input medicine details (name, time, dosage).
2. To implement a reminder mechanism using system time (clock-based logic).
3. To display alerts/notifications when it's time to take medicine.
4. To store medicine schedules using variables, arrays, or file handling (as used in your code).
5. To allow adding multiple medicine reminders in the system.
6. To compare current time with scheduled time using conditional statements.
7. To use loops for continuously checking reminder timings.
8. To create a simple user interface (console/menu-driven program).
9. To demonstrate use of core programming concepts like functions, conditions, and loops.
10. To ensure the system runs continuously until the user exits.

3. SYSTEM ANALYSIS

1. Problem Definition

Many users forget to take medicines on time due to busy schedules. There is no proper system to manage medicine schedules and give timely reminders. This leads to health risks and improper treatment

2. Existing System

- Users rely on memory or manual notes
- Mobile alarms can be set but cannot store medicine details properly
- No facility to manage multiple medicines in one place
- No permanent storage of medicine schedule

3. Proposed System

- Allows users to enter medicine name and time
- Stores data in a file (medicines.txt)
- Displays saved medicines
- Automatically checks system time and gives reminders using popup alerts
- Provides a simple graphical interface using Swing (JFrame)

4. System Requirements

a) Hardware Requirements

- Computer/Laptop
- Minimum 4GB RAM
- Basic processor

b) Software Requirements

- Programming Language: Java
- IDE: VS Code / IntelliJ / Eclipse
- JDK installed
- Operating System: Windows/Linux

5. Functional Requirements (Based on Code)

- Input medicine name using JTextField
- Input time in HH:mm format
- Add medicine using button (Add Medicine)
- Display medicines in JTextArea
- Store data in file using BufferedWriter

- Load saved data using `BufferedReader`
- Compare current time using `LocalTime`
- Show reminder using `JOptionPane` popup
- Run continuous background checking using `Thread`

6. Non-Functional Requirements

- User-friendly GUI interface
- Accurate time checking (minute-based)
- Fast performance
- Data persistence using file handling
- Reliable execution with minimal errors

7. Feasibility Study

a) Technical Feasibility

- Swing (GUI)
- File Handling
- Multithreading
- Time API (`LocalTime`)

b) Economic Feasibility

- No cost (uses free tools and software)

c) Operational Feasibility

- Easy to use for basic computer users
- Simple buttons and display

8. System Design Overview (Code Mapping)

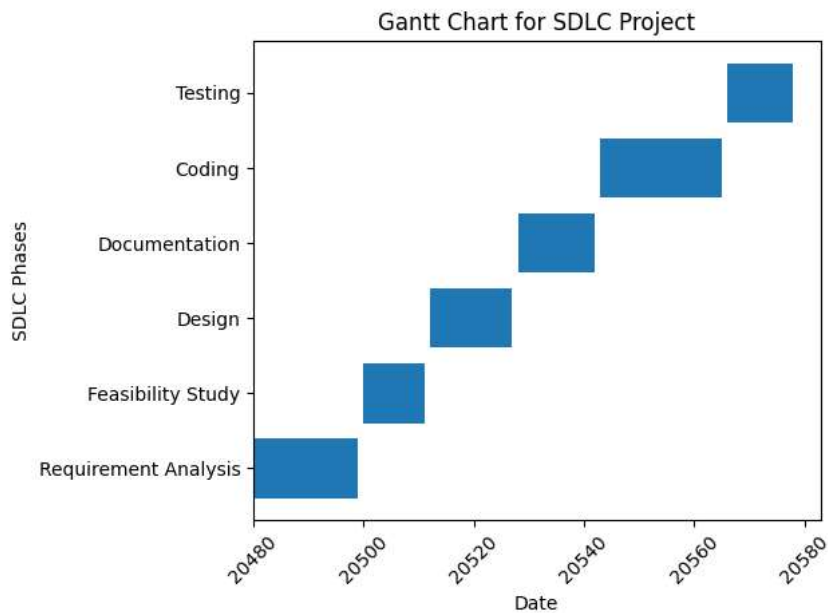
- **Class Medicine** → stores name and time
- **ArrayList** → stores multiple medicines
- **GUI Components** → input/output interface
- **File Handling** → permanent storage
- **Thread** → continuous reminder checking
- **Time API** → real-time comparison

9. Limitations

- Works only when program is running
- No mobile notifications (in basic version)

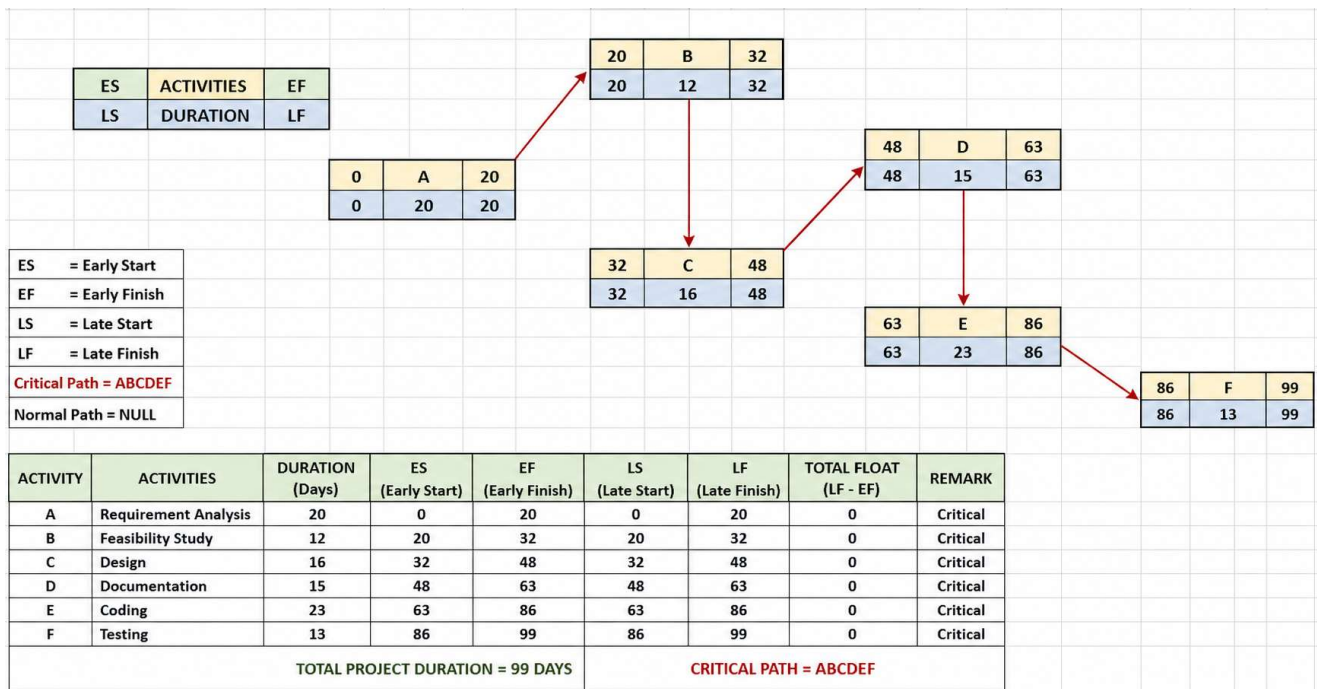
4.GANTT CHART

SDLC Phase	Project Activity	Duration	Start	Finish
1	Requirement Analysis	20 days	27-01-2026	15-02-2026
2	Feasibility Study	12 days	16-02-2026	27-02-2026
3	Design	16 days	28-02-2026	15-03-2026
4	Documentation	15 days	16-03-2026	30-03-2026
5	Coding	23 days	31-03-2026	22-04-2026
6	Testing	13 days	23-04-2026	05-05-2026



5.PERT CHART

SDLC Phase	Project Activity	Duration	Start	Finish
1	Requirement Analysis	20 days	27-01-2026	15-02-2026
2	Feasibility Study	12 days	16-02-2026	27-02-2026
3	Design	16 days	28-02-2026	15-03-2026
4	Documentation	15 days	16-03-2026	30-03-2026
5	Coding	23 days	31-03-2026	22-04-2026
6	Testing	13 days	23-04-2026	05-05-2026



6.REQUIREMENT SPECIFICATIONS

Technical Specification

Software Requirements:

- **Operating System** – Windows 10 / Windows 11
- **Progamming** – Java
- **Database** – MySQL Database
- **NotificationSystem** – The software should support a notification .
- **Local Server** – XAMPP or WAMP (for database connection)

Hardware Requirements:

- **Laptop Model** – Victus by HP
- **Processor** – AMD Ryzen5
- **RAM** – Minimum 16 GB (recommended for smooth performance)
- **Storage** – 512GB SSD (or as available)
- **Internet Connection** – Required for online hosting and updates

7.SYSTEM DESIGN

Modularization Details

The system is divided into independent modules for better maintainability and scalability.

1. User Management Module

- Registration & login
- Authentication (username/password)
- Profile management

Inputs : user details

Outputs: authenticated user session

2. Medicine Management Module

- Add/Edit/Delete medicines
- Store dosage, frequency, duration
- Handle multiple medicines per user

Inputs : medicine name , schedule

Outputs: stored medicine records

3. Database Module

- Store all persistent data
- Maintain relationships
- Ensure data consistency

4. Security Module

- Password encryption
- Secure access control
- Session management

Data Integrity and Constraints

1. Entity Integrity

- Each table must have a Primary Key (PK)
- Example:
 - user_id in Users
 - medicine_id in Medicines

Ensures no duplicate records

2. Referential Integrity

- Use Foreign Keys (FK) to maintain relationships

Example:

- `user_id` in Medicines → references Users
- `medicine_id` in Reminders → references Medicines

8.TESTING

Testing is the process of evaluating the system to ensure it meets specified requirements and works correctly without errors. For the Medicine Reminder System, testing ensures that reminders are accurate, timely, and reliable.

Objectives of Testing

- To verify that all functionalities work correctly
- To ensure reminders are triggered at the correct time
- To detect and fix errors or bugs
- To ensure smooth interaction between modules
- To check system performance and usability

Testing Strategy

The system was tested using a black-box testing approach, focusing on input and output without considering internal code structure. Both manual testing and functional testing were performed.

Types of Testing

Unit Testing

Each module was tested individually:

- Medicine entry module
- Reminder scheduling module
- Notification system

Ensured each unit works independently.

Integration Testing

- Combined modules were tested together
- Example: Medicine data + reminder alert system

Ensured proper data flow between modules.

System Testing

- Entire system tested as a whole
- Checked complete workflow:
- Add medicine → Set time → Receive reminder Ensured the system meets requirements.

User Interface Testing

- Checked buttons, forms, and navigation
- Verified ease of use for users

Ensured system is user-friendly.

Performance Testing

- Verified reminders are sent without delay
- Checked system response time

Ensured efficient performance.

9.SYSTEM SECURITY MEASURES

A Medicine Reminder System stores personal information such as user details, medicine schedules, dosage timings, and reminder history. Therefore, proper security measures are necessary to protect data confidentiality, integrity, and availability.

1. User Authentication

Authentication is used to verify the identity of users before allowing access to the system.

Security methods:

- Username and password login
- OTP verification during registration/login
- Biometric authentication (fingerprint or face unlock in mobile apps)

Purpose:

Ensures that only authorized users can access their medicine information.

2. Authorization and Access Control

Authorization controls what actions each user can perform after login.

Implementation:

- Normal users can manage only their own reminders and medicine records
- Admin can manage user accounts and monitor the system

Purpose:

Prevents users from accessing or modifying another user's data.

3. Password Security

Passwords should never be stored as plain text.

Measures:

- Password hashing
- Strong password requirements:

Purpose:

Protects user accounts even if the database is compromised.

4. Data Encryption

Sensitive information should be encrypted.

a) Encryption in Transit

b) Encryption at Rest

Purpose:

Prevents unauthorized reading of personal data.

5. Input Validation

All user inputs must be validated before processing.

Examples:

- Dosage cannot be negative
- Medicine name cannot be empty

Protection Against:

- SQL Injection
- Invalid data entry

Purpose:

Maintains system stability and data accuracy.

6. Session Management

Secure sessions are required after login.

Measures:

- Auto logout after inactivity
- Secure session tokens
- Session timeout

Purpose:

Prevents unauthorized access if device is left unattended.

7. Backup and Recovery

Regular backup is necessary to avoid data loss.

Methods:

- Daily database backup
- Cloud backup storage
- Recovery process after crash or failure
- Purpose:
Ensures continuity of service and data availability.

8. Notification Privacy

Medicine reminders may contain private health details.

Measures:

- Hide medicine details on lock screen

Purpose:

Protects user privacy from others viewing device notifications.

9. Logging and Monitoring

The system should maintain activity logs.

Logs include:

- Login attempts
- Password changes
- Reminder updates

Purpose:

Helps detect suspicious activities and security issues.

10. Firewall and Network Security

Server protection is required against external threats.

Measures:

- Firewall implementation
- Antivirus and malware protection
- Network monitoring

Purpose:

Protects the system from cyberattacks.

11. Software Updates and Patch Management

The application should be regularly updated.

Includes:

- Security patches
- Bug fixes
- Library updates

Purpose:

Protects system from newly discovered vulnerabilities.

12. Database Security

Database access must be restricted.

Measures:

- Role-based database access
- Backup encryption
- Limited admin privileges

Purpose:

Secures stored medicine and user records.

10.PROBLEM DEFINITION

1. Many people forget to take medicines on time.
2. Missing doses can cause serious health problems.
3. Manual methods like notes or memory are not reliable.
4. There is a need for an automatic reminder system.
5. The system will help users manage their medicine schedule.
6. It will provide timely alerts for taking medicines.
7. The goal is to improve health and ensure no dose is missed.

11.IDENTIFICATION OF NEED

1. In modern life, people have busy and irregular schedules, which makes it difficult to remember medicine timings consistently.
2. Elderly patients and individuals suffering from chronic diseases require strict adherence to medication schedules, but they often depend on others for reminders.
3. Missing doses, taking medicines at the wrong time, or overdosing can lead to serious health complications and reduce the effectiveness of treatment.
4. Traditional methods such as written notes, alarms, or memory-based tracking are not always reliable and can easily be ignored or forgotten.
5. There is a strong need for an automated and user-friendly system that can store medicine details and manage schedules efficiently.
6. The system should provide timely alerts or notifications to remind users exactly when to take their medicines.
7. Such a system will reduce dependency on manual tracking and help users maintain a proper medication routine.
8. Overall, the Medicine Reminder System is needed to improve health management, ensure timely medication, and minimize the risk of missed doses.

12.FEASIBILITY STUDY

1. Technical Feasibility

The project is technically feasible because it can be developed using basic programming languages like Java and simple databases such as SQLite or MySQL. The required tools and technologies are easily available and suitable for a BCA-level project. The alert system can also be implemented using built-in libraries.

2. Economic Feasibility

The project is cost-effective as it does not require expensive hardware or software. Most tools used (like IDEs and databases) are free or open-source. Therefore, the development and implementation cost is low.

3. Operational Feasibility

The system is easy to use and user-friendly. Even non-technical users can operate it without difficulty. It helps users manage their medicine schedule efficiently, making it practical for real-life use.

4. Schedule Feasibility

The project can be completed within a limited time (such as 30 days). All modules like design, coding, and testing can be finished within the given project timeline.

5. Legal Feasibility

The system does not violate any legal rules or regulations. It only stores basic user data and medicine schedules, ensuring privacy and safe usage.

13.PROJECT PLANNING

1. Project Objective

The objective of this project is to develop a system that reminds users to take medicines on time and helps manage their medication schedule efficiently.

2. Project Scope

- Add, update, and delete medicine details
- Set time-based reminders
- Provide alert/notification system
- View daily medicine schedule

3. Project Activities

- Requirement Analysis
Collect and analyze user requirements for the system.
- System Design
Design system architecture, database, and interface.
- Coding/Development
Develop the application using Java.
- Testing
Test the system to find and fix errors.
- Implementation
Deploy the system for user use.
- Documentation
Prepare project report and documentation.

4. Project Schedule

The project will be completed in approximately 30 days with proper planning and execution of all phases.

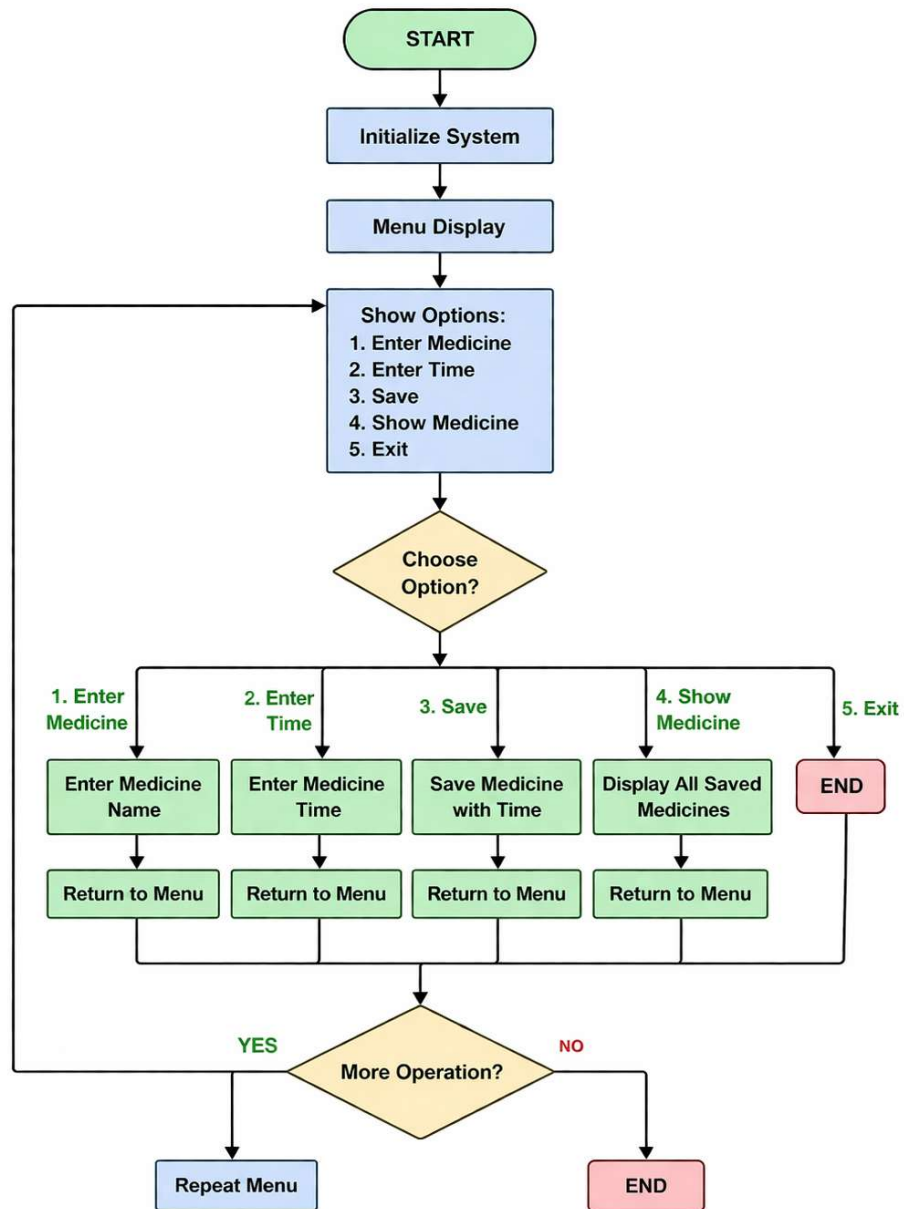
5. Resource Planning

- Hardware: Victus by HP
- Software: NetBeans or Eclipse IDE
- Database: MySQL or SQLite

6. Risk Management

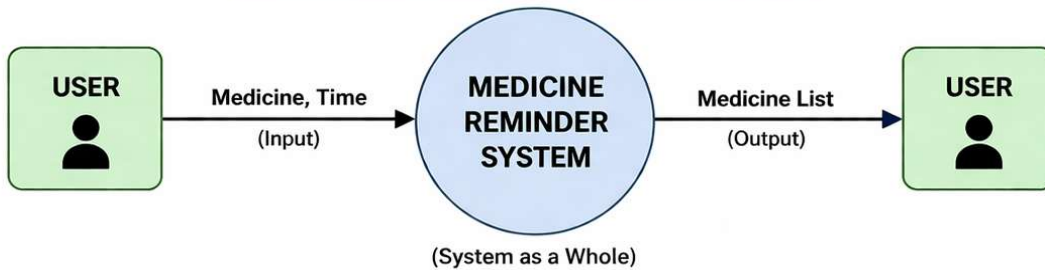
- Delay in development → Proper time management
- Bugs in system → Regular testing
- Data loss → Use database backup

14.FLOW CHART

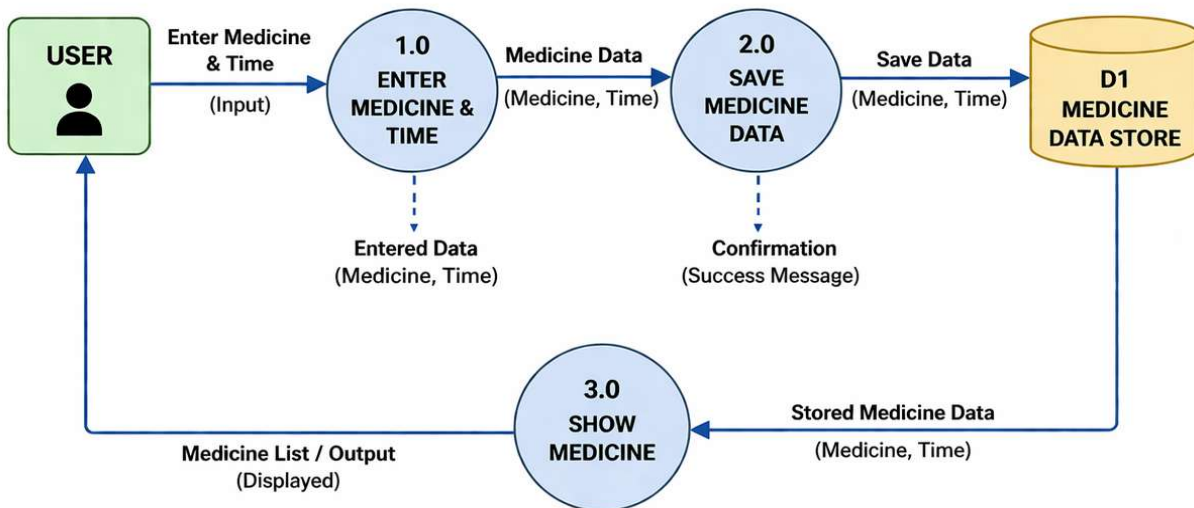


15.DATA FLOW DIAGRAM(DFD)

DFD LEVEL 0 (Context Diagram)



DFD LEVEL 1 (Detailed Diagram)



LEGEND:  External Entity  Process  Data Store  Data Flow

16.SCOPE

1. The system will allow users to add, update, and delete medicine details such as name, dosage, and time.
2. It will provide a reminder feature to notify users at the correct time to take medicines.
3. The system will maintain a daily schedule of medicines for easy tracking.
4. It will help users avoid missing doses and improve medication management.
5. The application can be used by individuals, especially elderly people and patients with regular medication needs.
6. The system will be simple, user-friendly, and easy to operate.
7. The project is limited to basic reminder functionality and does not include advanced features like doctor integration or online consultation.

17.CODE

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
import java.io.*;
import java.time.*;
import java.time.format.DateTimeFormatter;
import java.util.*;

class Medicine {
    String name;
    String time;

    Medicine(String name, String time) {
        this.name = name;
        this.time = time;
    }
}

public class AdvancedMedicineReminder extends JFrame {

    private JTextField nameField, timeField;
    private JTextArea displayArea;
    private ArrayList<Medicine> medicineList = new ArrayList<>();
    private final String FILE_NAME = "medicines.txt";

    public AdvancedMedicineReminder() {
        setTitle("Medicine Reminder System");
        setSize(400, 400);
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setLayout(new FlowLayout());
    }
}
```

```
JLabel nameLabel = new JLabel("Medicine Name:");
nameField = new JTextField(20);

JLabel timeLabel = new JLabel("Time (HH:mm):");
timeField = new JTextField(10);

JButton addButton = new JButton("Add Medicine");
JButton showButton = new JButton("Show Medicines");

displayArea = new JTextArea(10, 30);
displayArea.setEditable(false);

add(nameLabel);
add(nameField);
add(timeLabel);
add(timeField);
add(addButton);
add(showButton);
add(new JScrollPane(displayArea));

loadFromFile();

addButton.addActionListener(e -> addMedicine());
showButton.addActionListener(e -> showMedicines());

startReminder();

setVisible(true);
}
```

```

private void addMedicine() {
    String name = nameField.getText();
    String time = timeField.getText();

    if (name.isEmpty() || time.isEmpty()) {
        JOptionPane.showMessageDialog(this, "Please fill all fields!");
        return;
    }

    medicineList.add(new Medicine(name, time));
    saveToFile();

    JOptionPane.showMessageDialog(this, "Medicine Added!");
    nameField.setText("");
    timeField.setText("");
}

private void showMedicines() {
    displayArea.setText("");
    for (Medicine m : medicineList) {
        displayArea.append(m.name + " at " + m.time + "\n");
    }
}

private void saveToFile() {
    try (BufferedWriter bw = new BufferedWriter(new FileWriter(FILE_NAME))) {
        for (Medicine m : medicineList) {
            bw.write(m.name + "," + m.time);
            bw.newLine();
        }
    } catch (IOException e) {

```

```

        e.printStackTrace();
    }
}

private void loadFromFile() {
    try (BufferedReader br = new BufferedReader(new FileReader(FILE_NAME))) {
        String line;
        while ((line = br.readLine()) != null) {
            String[] data = line.split(",");
            medicineList.add(new Medicine(data[0], data[1]));
        }
    } catch (IOException e) {

    }
}

private void startReminder() {
    Thread t = new Thread(() -> {
        DateTimeFormatter formatter = DateTimeFormatter.ofPattern("HH:mm");

        while (true) {
            String now = LocalDateTime.now().format(formatter);

            for (Medicine m : medicineList) {
                if (m.time.equals(now)) {
                    JOptionPane.showMessageDialog(this,
                        "🕒 Take Medicine: " + m.name);
                }
            }
        }

        try {

```

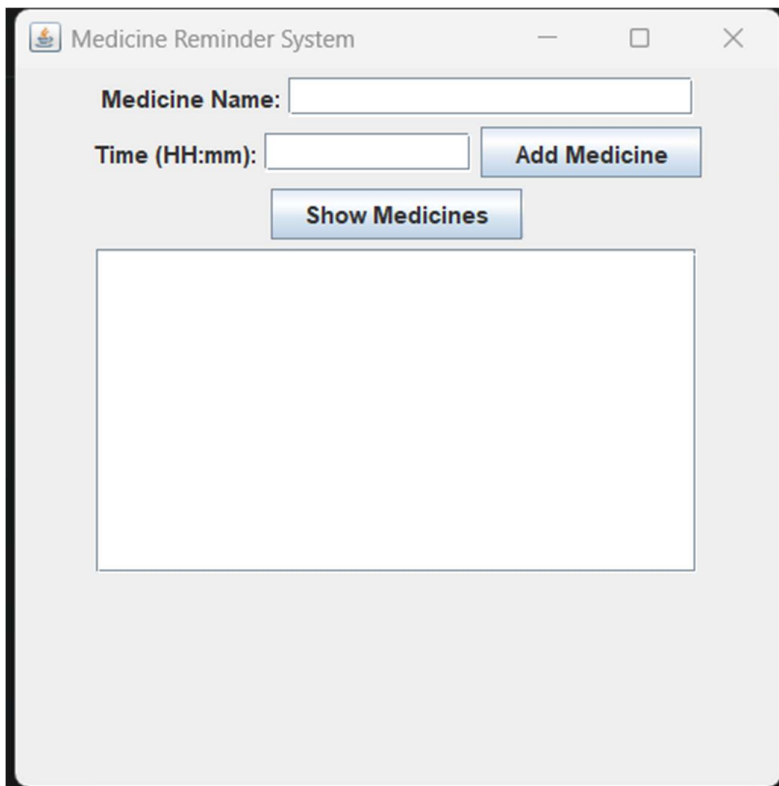
```
        Thread.sleep(60000);
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
}

});

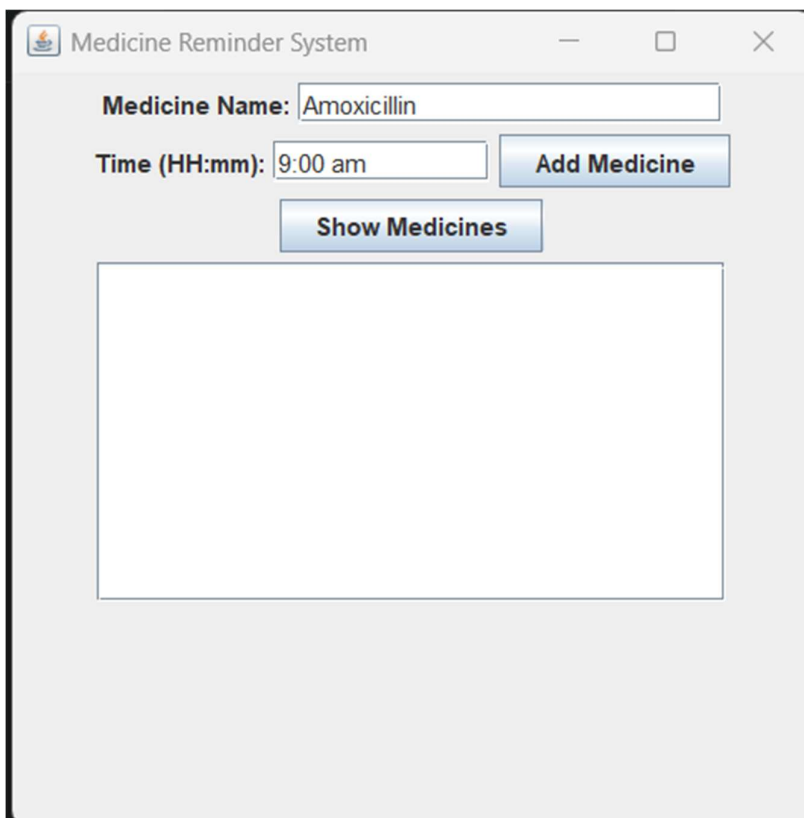
t.start();
}

public static void main(String[] args) {
    new AdvancedMedicineReminder();
}
}
```

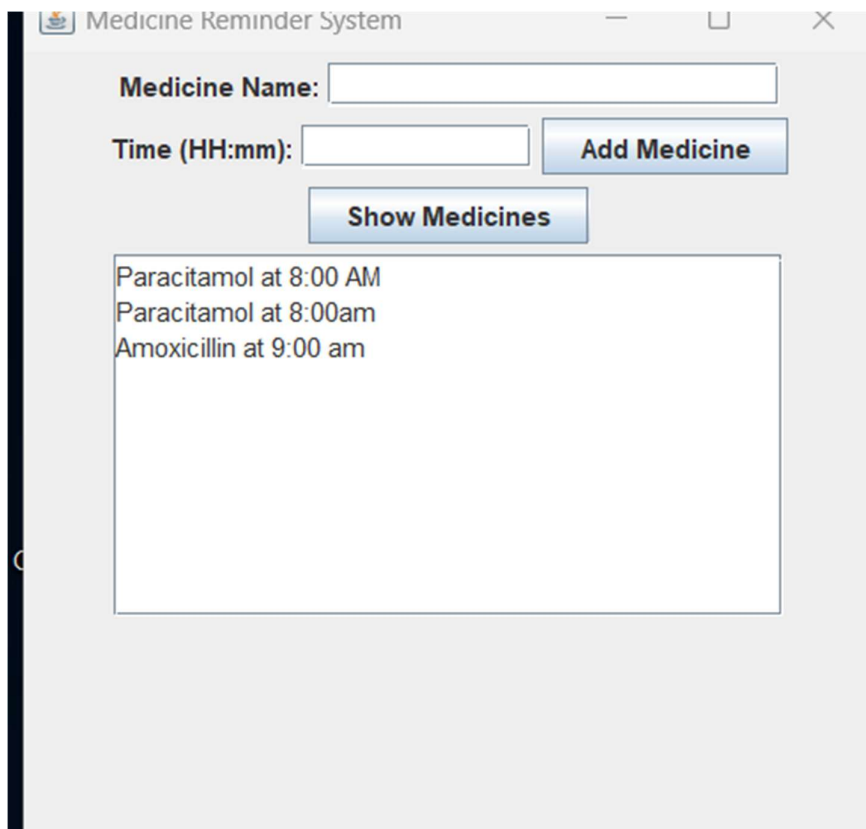
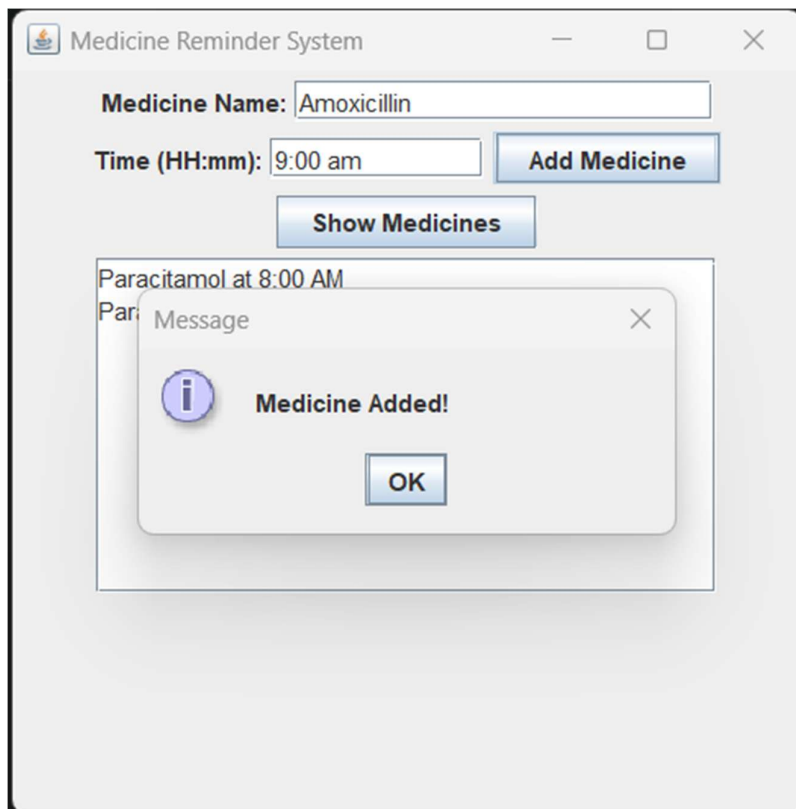
Output:



The screenshot shows a window titled "Medicine Reminder System". It contains two input fields: "Medicine Name:" and "Time (HH:mm):", both of which are empty. To the right of the "Time (HH:mm):" field is a blue button labeled "Add Medicine". Below the "Time (HH:mm):" field is another blue button labeled "Show Medicines". At the bottom of the window is a large, empty rectangular box, likely intended for displaying a list of medicines.



The screenshot shows the same "Medicine Reminder System" window, but now the input fields are populated. The "Medicine Name:" field contains the text "Amoxicillin", and the "Time (HH:mm):" field contains the text "9:00 am". The "Add Medicine" and "Show Medicines" buttons remain visible. The large empty box at the bottom is still present.



18.BIBLIOGRAPHY

1. ChatGPT – <https://chat.openai.com>
2. Google – <https://www.google.com>
3. GeeksforGeeks – <https://www.geeksforgeeks.org>
4. W3Schools – <https://www.w3schools.com>
5. TutorialsPoint – <https://www.tutorialspoint.com>
6. Stack Overflow – <https://stackoverflow.com>